

PRAKTIKUM SISTEM OPERASI

MODUL 11

Memori Xinu



Oleh:

Haza Zaidan Zidna Fann

2311104056

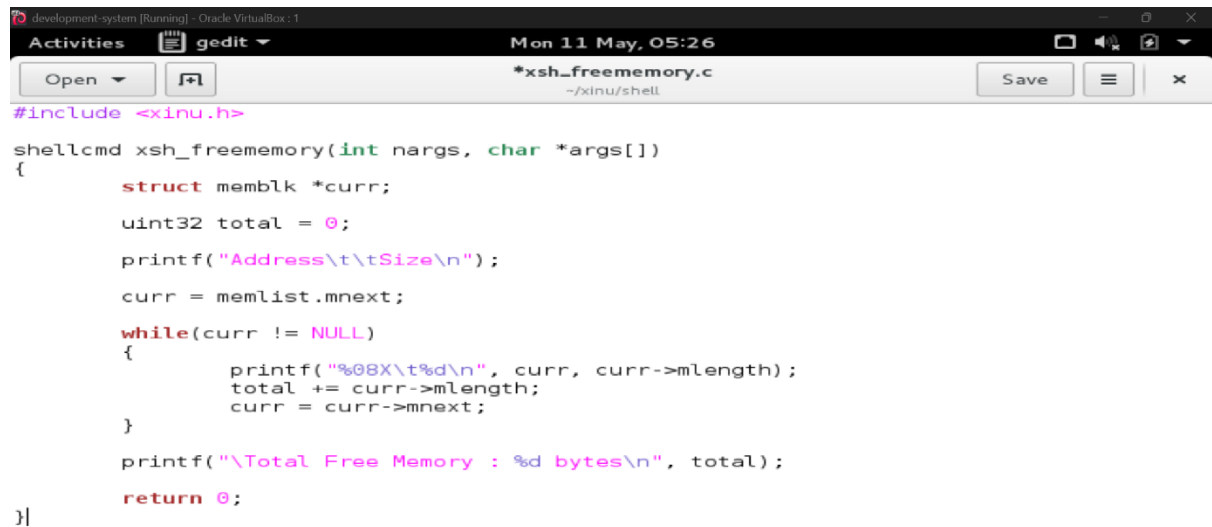
Program Studi S1 Rekayasa Perangkat Lunak

DIREKTORAT KAMPUS PURWOKERTO UNIVERSITAS TELKOM

2026

Jurnal

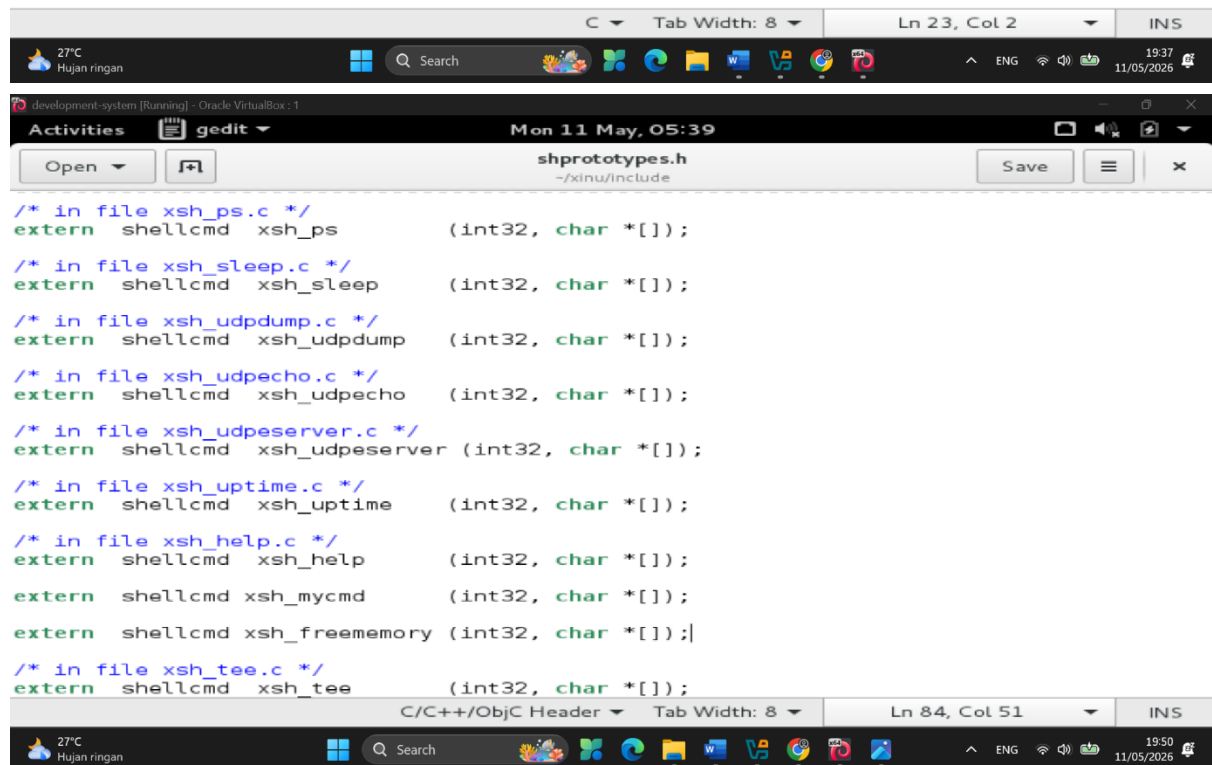
1.



The screenshot shows a gedit editor window titled "development-system [Running] - Oracle VirtualBox : 1". The window displays the implementation of the `xsh_freememory` function in the file `*xsh_freememory.c`. The code is as follows:

```
#include <xinu.h>

shellcmd xsh_freememory(int nargs, char *args[])
{
    struct memblk *curr;
    uint32 total = 0;
    printf("Address\t\tSize\n");
    curr = memlist.mnext;
    while(curr != NULL)
    {
        printf("%08X\t%d\n", curr, curr->mlength);
        total += curr->mlength;
        curr = curr->mnext;
    }
    printf("\Total Free Memory : %d bytes\n", total);
    return 0;
}
```



The screenshot shows a gedit editor window titled "development-system [Running] - Oracle VirtualBox : 1". The window displays the declaration of shellcmd functions in the file `shprototypes.h`. The code is as follows:

```
/* in file xsh_ps.c */
extern shellcmd xsh_ps (int32, char *[]);

/* in file xsh_sleep.c */
extern shellcmd xsh_sleep (int32, char *[]);

/* in file xsh_udpdump.c */
extern shellcmd xsh_udpdump (int32, char *[]);

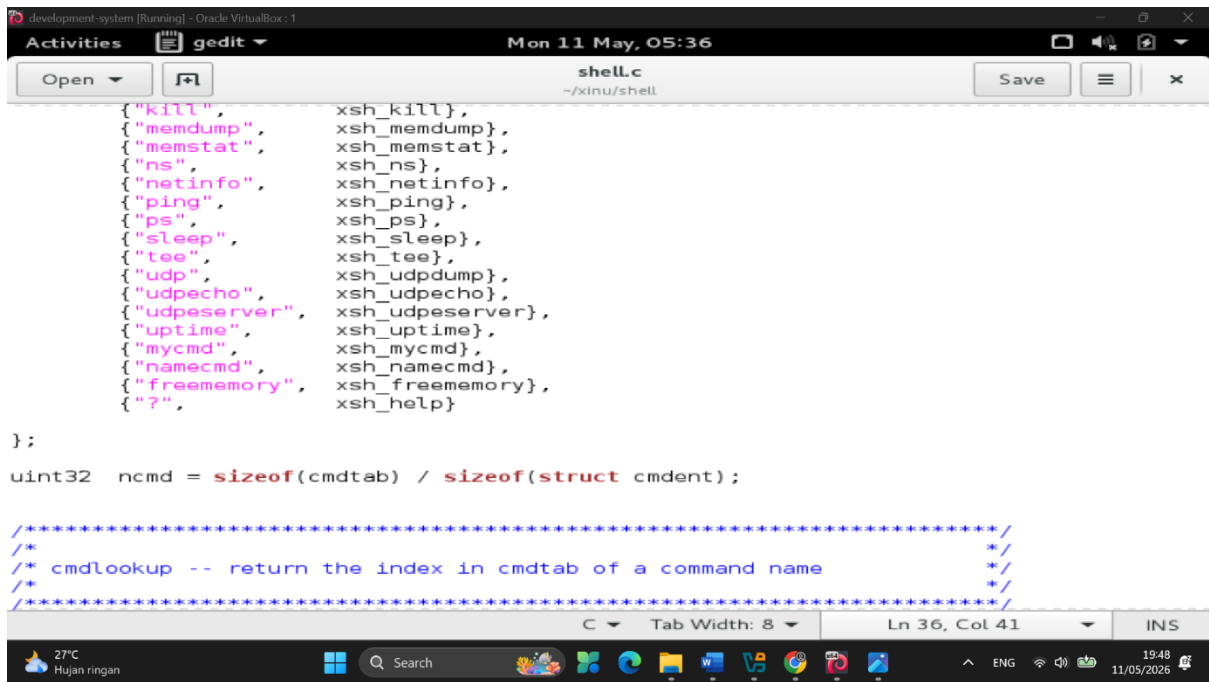
/* in file xsh_udpecho.c */
extern shellcmd xsh_udpecho (int32, char *[]);

/* in file xsh_udpserver.c */
extern shellcmd xsh_udpserver (int32, char *[]);

/* in file xsh_uptime.c */
extern shellcmd xsh_uptime (int32, char *[]);

/* in file xsh_help.c */
extern shellcmd xsh_help (int32, char *[]);
extern shellcmd xsh_mycmd (int32, char *[]);
extern shellcmd xsh_freememory (int32, char *[]);

/* in file xsh_tee.c */
extern shellcmd xsh_tee (int32, char *[]);
```



```
development-system [Running] - Oracle VirtualBox : 1
Activities gedit Mon 11 May, 05:36
shell.c
~/xinu/shell

{"kill", xsh_kill},
{"memdump", xsh_memdump},
{"memstat", xsh_memstat},
{"ns", xsh_ns},
{"netinfo", xsh_netinfo},
{"ping", xsh_ping},
{"ps", xsh_ps},
{"sleep", xsh_sleep},
{"tee", xsh_tee},
{"udp", xsh_udpdump},
{"udpecho", xsh_udpecho},
{"udpserver", xsh_udpserver},
{"uptime", xsh_uptime},
{"mycmd", xsh_mycmd},
{"namecmd", xsh_namecmd},
{"freememory", xsh_freememory},
{"?", xsh_help}

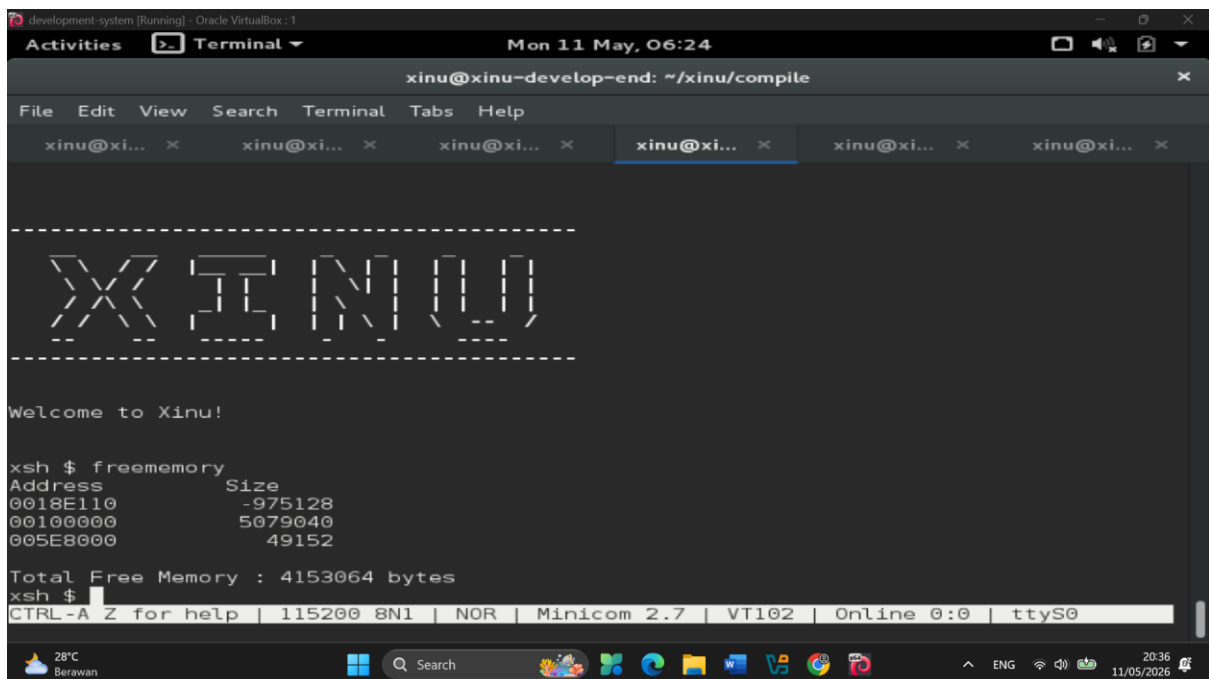
};

uint32 ncmd = sizeof(cmdtab) / sizeof(struct cmdent);

/*****
/*
/* cmdlookup -- return the index in cmdtab of a command name
/*
*****/
```

Address pada output freememory menunjukkan alamat awal dari blok memori kosong yang tersedia di sistem Xinu. Alamat ini digunakan sistem operasi untuk mengetahui lokasi memori yang masih dapat dipakai oleh proses atau program lain. Setiap baris address merepresentasikan satu blok memori bebas yang tersimpan dalam linked list memlist, sehingga Xinu dapat mengatur dan mengalokasikan memori dengan lebih terstruktur.

2.



```
development-system [Running] - Oracle VirtualBox : 1
Activities Terminal Mon 11 May, 06:24
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Tabs Help
xinu@xi... xinu@xi... xinu@xi... xinu@xi... xinu@xi... xinu@xi...

XINU

Welcome to Xinu!

xsh $ freememory
Address      Size
0018E110     -975128
00100000      5079040
005E8000       49152

Total Free Memory : 4153064 bytes
xsh $

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS0
```

Size menunjukkan besar kapasitas dari setiap blok memori kosong yang terdapat pada address tersebut. Nilai size dihitung dalam satuan byte dan digunakan untuk mengetahui seberapa banyak memori yang masih tersedia pada blok itu. Pada program freememory,

nilai size diambil dari `curr->mlength`, lalu seluruh size dijumlahkan untuk menghasilkan Total Free Memory, yaitu total keseluruhan memori kosong yang masih tersedia di sistem.

3.

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan data segment dan BSS segment untuk mengatur penggunaan memori secara lebih efisien. Data segment digunakan untuk menyimpan variabel global atau static yang sudah memiliki nilai awal (initialized), sedangkan BSS segment digunakan untuk variabel global atau static yang belum diinisialisasi. Dengan pemisahan ini, variabel pada BSS tidak perlu disimpan lengkap di file executable karena sistem cukup menginisiasinya dengan nilai nol saat program dijalankan, sehingga ukuran program menjadi lebih kecil dan penggunaan memori lebih optimal.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Selama program berjalan, alokasi memori akan mengurangi ukuran free space karena sebagian memori digunakan oleh proses atau program yang aktif. Sebaliknya, dealokasi memori akan menambah kembali ukuran free space karena memori yang sebelumnya dipakai dikembalikan ke sistem. Pada Xinu, perubahan ini dikelola menggunakan free memory list sehingga setiap perubahan alokasi dan dealokasi akan memengaruhi jumlah dan ukuran block memori kosong yang tersedia.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap lebih berpotensi menimbulkan masalah karena pengelolaannya dilakukan secara dinamis oleh programmer atau sistem selama program berjalan. Kesalahan seperti lupa melakukan dealokasi dapat menyebabkan memory leak, sedangkan alokasi dan dealokasi yang tidak teratur dapat menimbulkan fragmentasi memori. Berbeda dengan stack yang pengelolaannya otomatis dan terstruktur berdasarkan urutan pemanggilan fungsi, sehingga lebih aman dan jarang menyebabkan masalah pengelolaan memori.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Xinu menggunakan linked list untuk menyimpan free block karena struktur ini fleksibel dalam menangani perubahan ukuran dan jumlah block memori kosong secara dinamis. Dengan linked list, sistem dapat dengan mudah menambahkan, menghapus, atau menggabungkan block memori tanpa harus memindahkan data lain di memori. Selain itu, linked list memudahkan traversal saat sistem mencari block kosong yang sesuai untuk proses alokasi memori.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Tantangan utama penggunaan heap di Xinu adalah fragmentasi memori dan pengelolaan alokasi yang efisien. Fragmentasi terjadi ketika banyak block kecil kosong tersebar

sehingga sulit digunakan untuk alokasi memori berukuran besar. Selain itu, sistem harus memastikan bahwa setiap alokasi dan dealokasi dilakukan dengan benar agar tidak terjadi memory leak atau kerusakan struktur free memory list yang dapat menyebabkan sistem menjadi tidak stabil.